Dzisiaj poruszamy następujące zagadnienia z sylabusa:

- **(9) Kopce.**
  Podstawowe operacje na kopcach binarnych, sortowanie przez kopcowanie, implementacja kolejki priorytetowej.
- **(10) Drzewa wyszukiwań binarnych.**
  Podstawowe własności, operacje słownikowe, metody przechodzenia drzewa BST.
- **(11) Algorytmy grafowe.**
  Przeszukiwanie grafu wszerz i wgłąb, znajdowanie najkrótszych ścieżek.

Grafy

Graf to wierzchołki i krawędzie między wierzchołkami:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) \qquad \leftarrow \text{graf to para zbiorów } \mathcal{V} \text{ i } \mathcal{E}$$

- Zbiór $\mathcal{V}$ to skończony zbiór **wierzchołków** – abstrakcyjnych etykiet (które możemy wyobrażać sobie jako punkty).
- Zbiór $\mathcal{E} \subseteq \{\{u, v\} \mid u, v \in \mathcal{V}\}$ to zbiór **krawędzi**, czyli dwuelementowych zbiorów definiujących pary wierzchołków.

Zwykły graf:

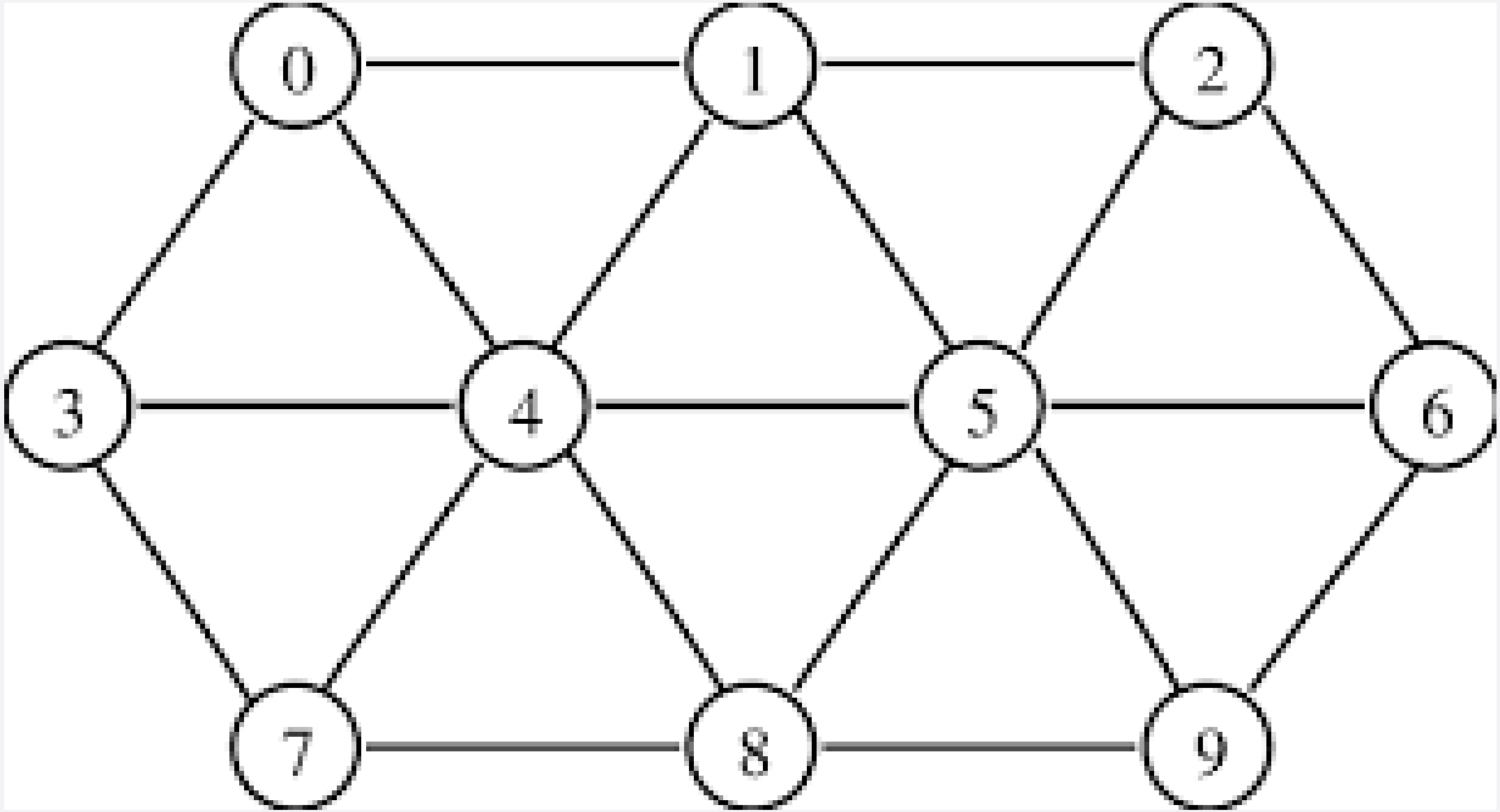
Pary wierzchołków $\{u, v\}$ i $\{v, u\}$ definiują tę samą krawędź:

$$u \longleftrightarrow v$$

Graf skierowany:

Pary wierzchołków (u, v) i (v, u) definiują krawędzie o przeciwnym skierowaniu:

$$(u, v) \equiv u \to v \qquad (v, u) \equiv v \to u$$


0
1
2
3
4
5
6
7
8
9

Przykłady grafów

Sieć znajomych – *graf nieskierowany*

- Każdy wierzchołek reprezentuję osobę w grupie znajomych.
- Krawędź $X \to Y$ oznacza, że osoba Y jest znajomą osoby X.

Naturalne założenie: jeśli X jest znajomym Y, to Y jest znajomym X.
$\Rightarrow$ Krawędzie są **symetryczne**, graf nie jest skierowany.

Odnośniki w internecie – *graf skierowany*

- Każdy wierzchołek reprezentuje stronę internetową ze zbioru stron (np. artykuły Wikipedii).
- Krawędź $X \to Y$ oznacza, że strona X zawiera odnośnik do strony Y.

Załóżmy, że $X \to Y$. Czy wówczas koniecznie $Y \to X$? **Z reguły nie**.
$\Rightarrow$ Krawędzie mają kierunki, graf jest skierowany.

Grafy ważone

Graf ważony to graf (skierowany lub nieskierowany), w którym każda krawędź ma przypisaną **wagę**. Waga krawędzi $u \to v$ definiuje *koszt*, który *płacimy*, przechodząc z u do v.

Na przykład: dystans między miastami u i v, cena biletu kolejowego, czas podróży, etc.

Formalnie:

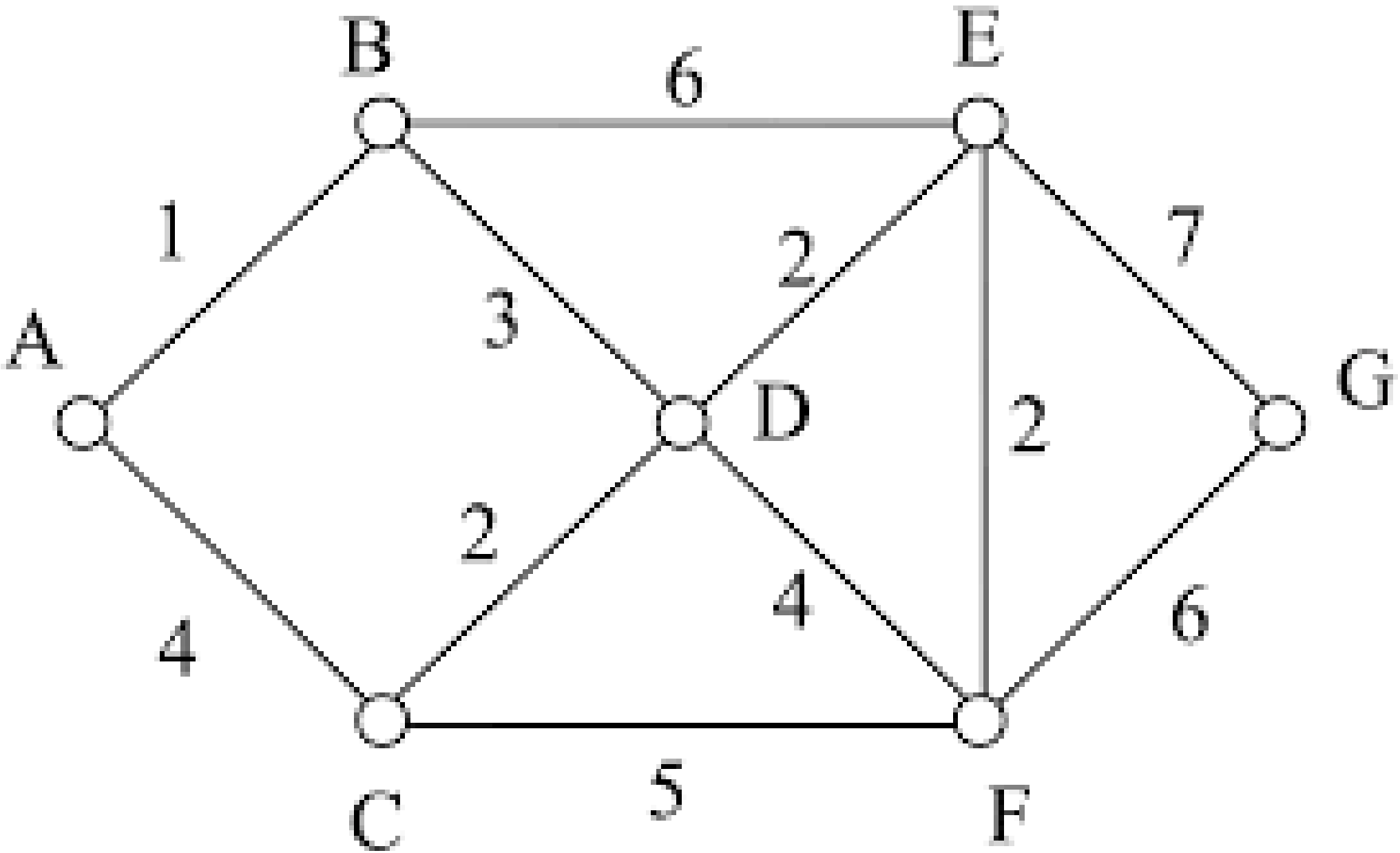
$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, w) \qquad \leftarrow \text{dwa zbiory i funkcja}$$

Funkcja $w : \mathcal{E} \to \mathbb{R}$ każdej krawędzi $e \in \mathcal{E}$ przypisuje wagę. Na przykład, napis

$$w(\{u, v\}) = x$$

oznacza, że krawędź łącząca $u \leftrightarrow v$ ma wagę równą x.

W ogólności, wagi mogą być liczbami dodatnimi, ujemnymi, ułamkami, liczbami niewymiernymi – ale w problemach zwykle rozważamy wagi, które są liczbami naturalnymi: $1, 2, 3, \ldots$


B
6
E
1
2
7
3
A
D
2
G
2
4
6
4
C
5
F

Problemy w analizie grafów

Które wierzchołki są osiągalne?

Załóżmy, że znajdujemy się w wierzchołku $v \in \mathcal{V}$ grafu skierowanego. Czy możemy znaleźć drogę do wierzchołka $u \in \mathcal{V}$ idąc po dozwolonych krawędziach?

Na przykład:

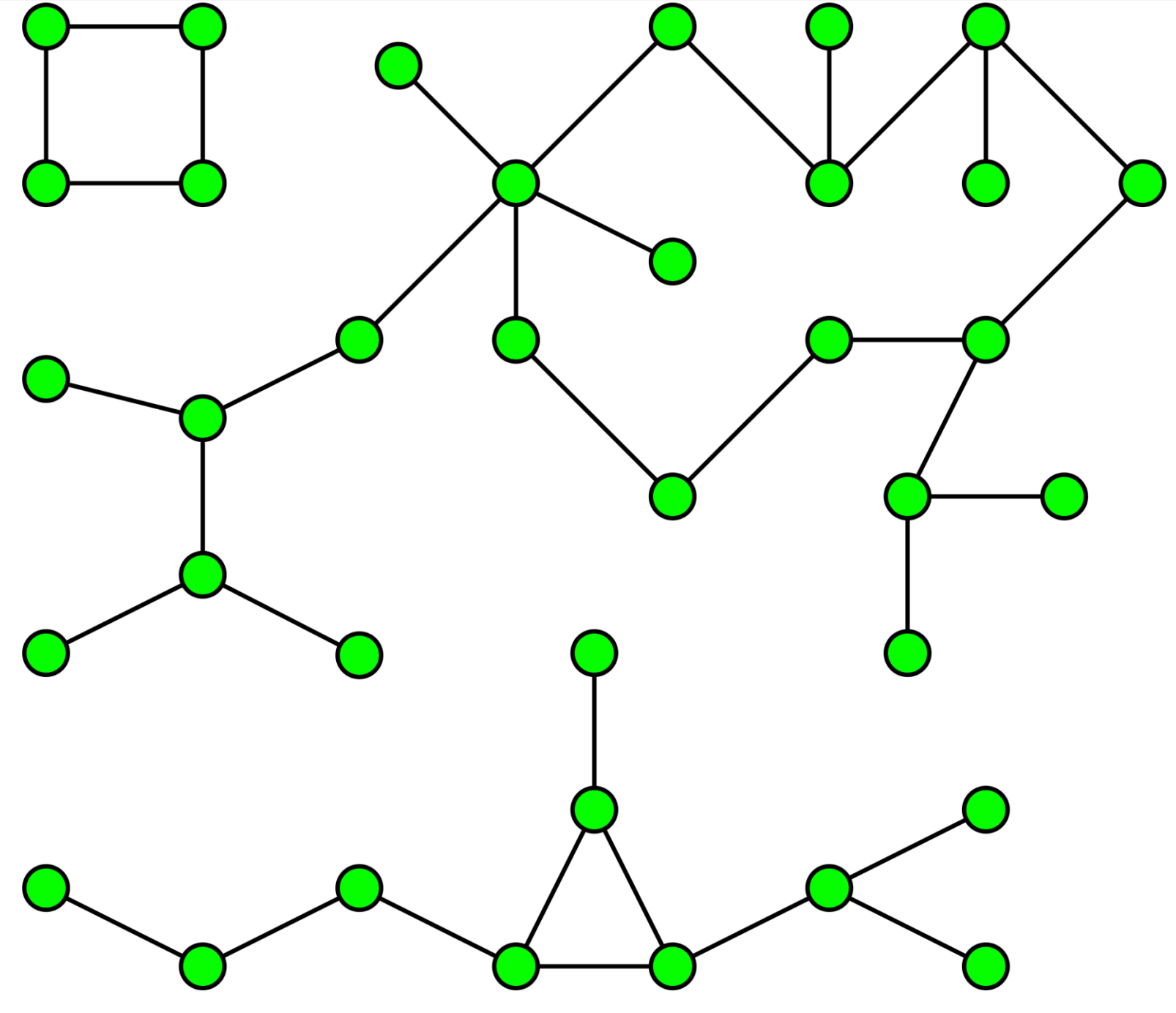
- Czy zaczynając na stronie X na Wikipedii, możemy dojść do strony Y klikając jedynie w odnośniki?
- Jeśli tak, to jaka jest minimalna liczba stron, które musimy odwiedzić po drodze?

Czy graf zawiera cykl?

Czy możemy znaleźć taką ścieżkę zaczynającą się i kończącą się w tym samym wierzchołku $v \in \mathcal{V}$, że każdy wierzchołek po drodze odwiedzamy dokładnie raz?

Na przykład:

- Czy z przystanku X możemy *odjechać i wrócić*, nie powtarzając przystanków?



Prosta implementacja struktury grafu

```python
class Graph:
    def __init__(self, n: int, directed: bool=False):
        # liczba wierzchołków
        self.n = n

        # czy graf jest skierowany
        self.directed = directed

        # lista sąsiedztwa - zainicjalizowana jako pusta
        self.adj = [[] for _ in range(n)]

    def add_edge(self, u: int, v: int):
        # dodaje krawędź między wierzchołkami u i v
        self.adj[u].append(v)

        # jeśli graf nie jest skierowany
        if not self.directed:
            # dodaje krawędź w drugą stronę
            self.adj[v].append(u)

    def neighbors(self, u: int):
        # zwraca listę sąsiadów wierzchołka u
        return self.adj[u]
```

Dla ułatwienia:

- Wierzchołki etykietujemy liczbami naturalnymi.
- Z góry zakładamy całkowitą liczbę wierzchołków równą `n`.

Sąsiedztwo w grafie modelujemy za pomocą listy list:

- Każdy element list `adj` sam jest listą.
- Sąsiedzi wierzchołka `u` są przechowywani w liście `adj[u]`.
- Żeby dodać nową krawędź ($u \to v$), umieszczamy `v` w liście `adj[u]`.
- Jeśli graf nie jest skierowany, to umieszczamy też `u` w liście `adj[v]`.

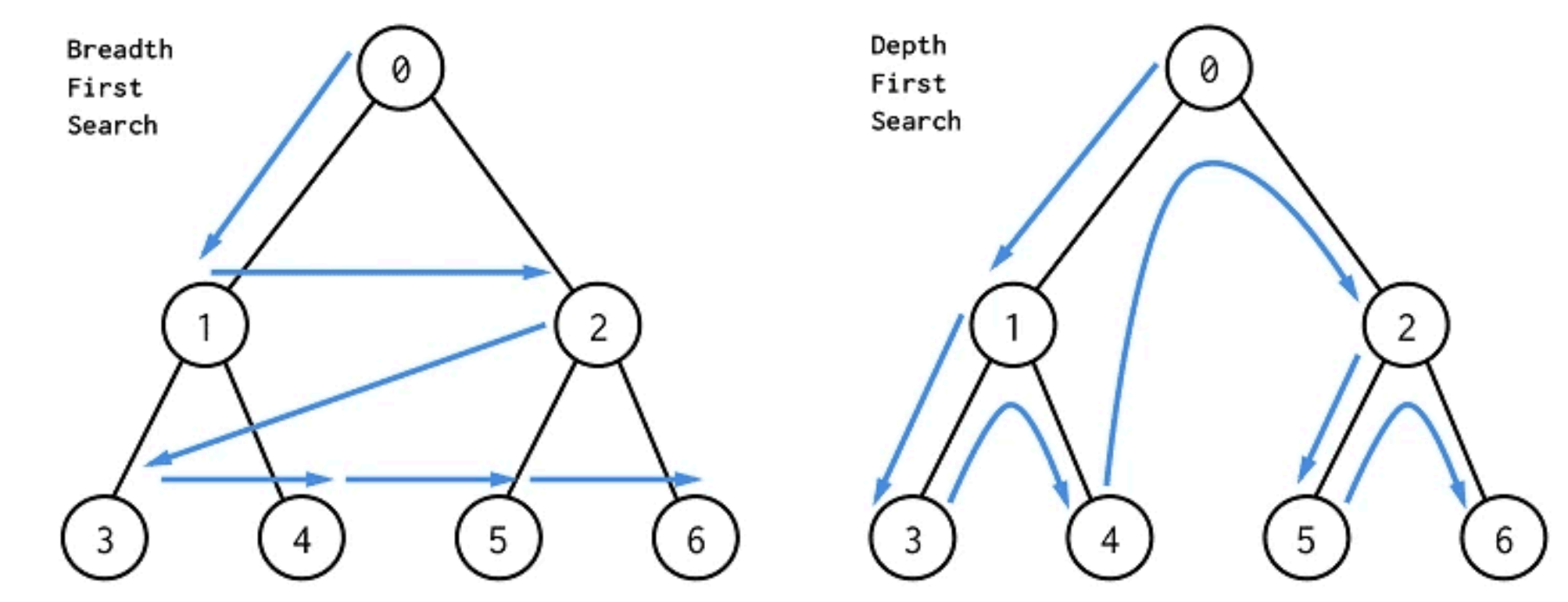
Przeszukiwanie drzewa

W grafie mamy skomplikowane relacje, możemy napotkać cykle. Łatwiej zacząć od *drzew*.

Istnieją dwa podstawowe algorytmy:

Przeszukiwanie wszerz – odwiedzamy wierzchołki warstwa po warstwie, zaczynając od korzenia.

Przeszukiwanie wgłąb – odwiedzamy kolejne dzieci, aż dojdziemy do liści.

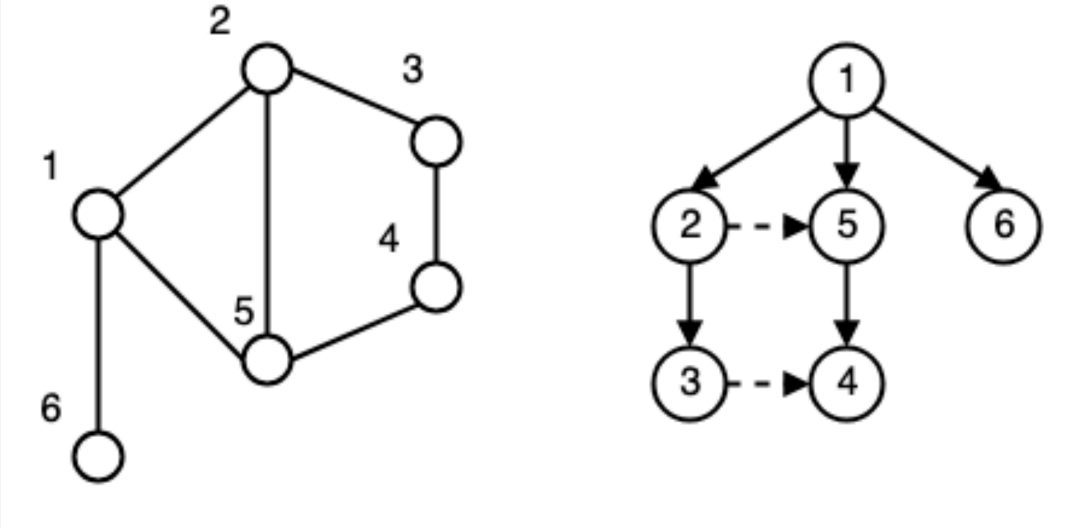


Przeszukiwanie grafu wszerz

Algorytm BFS dla grafu jest *niemalże identyczny*, ale:

- Musimy zdecydować, z którego wierzchołka zaczynamy (dla drzewa zaczynamy od korzenia).
- Musimy oznaczać, które wierzchołki już odwiedziliśmy.

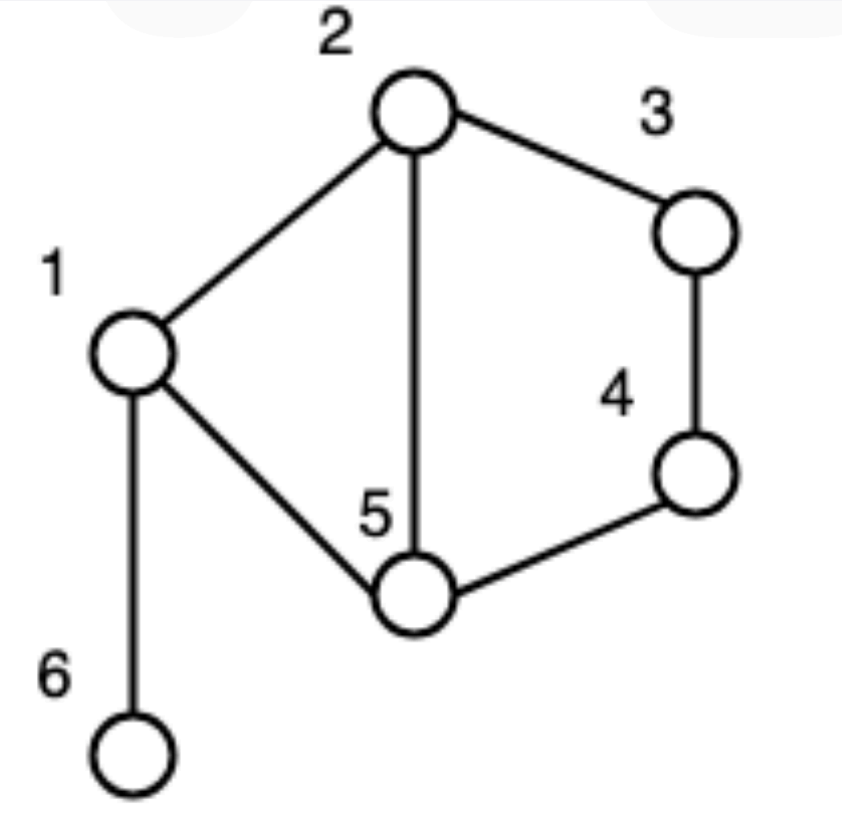
Wejście: graf + wierzchołek startowy `start`.
Wyjście: tablica odległości ze `start` do innych wierzchołków; tablica *poprzedników* (która buduje drzewo).



Przeszukiwanie wszerz wykorzystuje **kolejkę**.

1. Załóżmy, że zaczynamy w `1`:
   - Odległość od `1` do `1` wynosi 0, nie ma rodzica (piszemy, że rodzic to `-1`).
   - Zaznaczamy jako odwiedzony.
   - Patrzymy na sąsiadów: `2`, `5` i `6`. Oznaczamy jako odwiedzone, odległość to 0 + 1 = 1, poprzednik/rodzic to `1`. Kolejkujemy.
2. Patrzymy na kolejkę: pierwsze jest `2`.
   - Sąsiedzi `2` to `1`, `3` i `5`, ale `1` i `5` już odwiedziliśmy.
   - Patrzymy na `3`:
     - Odległość to `dist[2]` + 1 = 2.
     - Poprzednik to `2`.
   - Kolejkujemy `3`.
3. Patrzymy na kolejkę: pierwsze jest `5`.
   - Sąsiedzi `5` to `1`, `2` i `4`, ale `1` i `2` już odwiedziliśmy.
   - Patrzymy na `4`:
     - Odległość to `dist[5]` + 1 = 2.
     - Poprzednik to `5`.
   - Kolejkujemy `4`.

I tak dalej – dopóki kolejka nie jest pusta.



Przeszukiwanie wszerz pozwala znaleźć najkrótszą ścieżkę między wierzchołkami w grafie nieważonym `g`:

1. Chcemy najkrótszą ścieżkę startującą ze `start`, kończącą się w `target`.
2. Puszczamy `bfs(g, start)` – dostajemy tablice `dist` i `parent`.
3. Jeśli takiej ścieżki w ogóle nie ma, to `dist[target] == -1` – kończymy tutaj.
4. W przeciwnym razie będziemy korzystać z tablicy `parent`, żeby *wrócić* z `target` do `start`:
   - Zaczynamy z `v := target`. Definiujemy pustą listę `path = []`.
   - Jeżeli `v == start`, to zrekonstruowaliśmy ścieżkę.
   - Jeżeli `v != start`, to dodajemy `v` do listy `path` – `path.append(v)`.
   - Idzemy do rodzica: `v := parent[v]`.
5. Na końcu odwracamy `path` (bo zaczęliśmy od `target`) – `path.reverse()`.

Zadanie dla Państwa:

Przerobić algorytm DFS dla grafów na podstawie źródeł + kodu (w pliku `.ipynb`, który udostępniłem).

Struktura algorytmu jest *niemalże* identyczna – kolejkę zastępujemy przez stos.

Kopce

Kopiec to *prawie pełne drzewo binarne*:

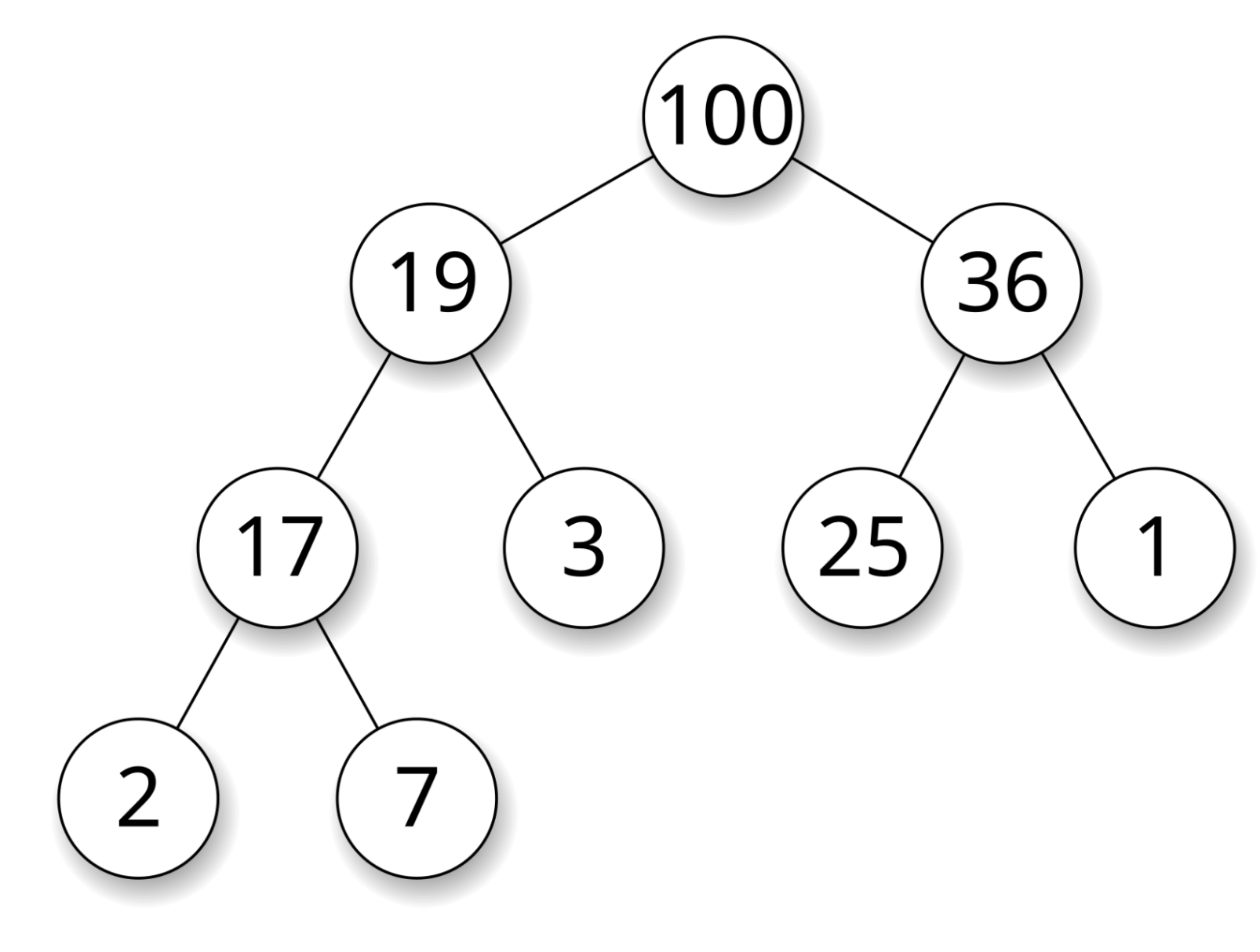
- Każdy wierzchołek przechowuje liczbę i ma dokładnie dwoje dzieci.
- Wszystkie poziomy drzewa są wypełnione od lewej do prawej.
- Ostatni poziom może być niepełny, ale wypełniamy też od lewej do prawej.

To warunki pozwalają nam przechowywać kopiec w tablicy.

Mamy dwa rodzaje kopców:

- **Max-heap:** każdy rodzic $\geq$ każde dziecko.
- **Min-heap:** każdy rodzic $\leq$ każde dziecko.

Stąd wynika, że korzeń przechowuje największą/najmniejszą wartość w całym drzewie.


100
19
36
17
3
25
1
2
7

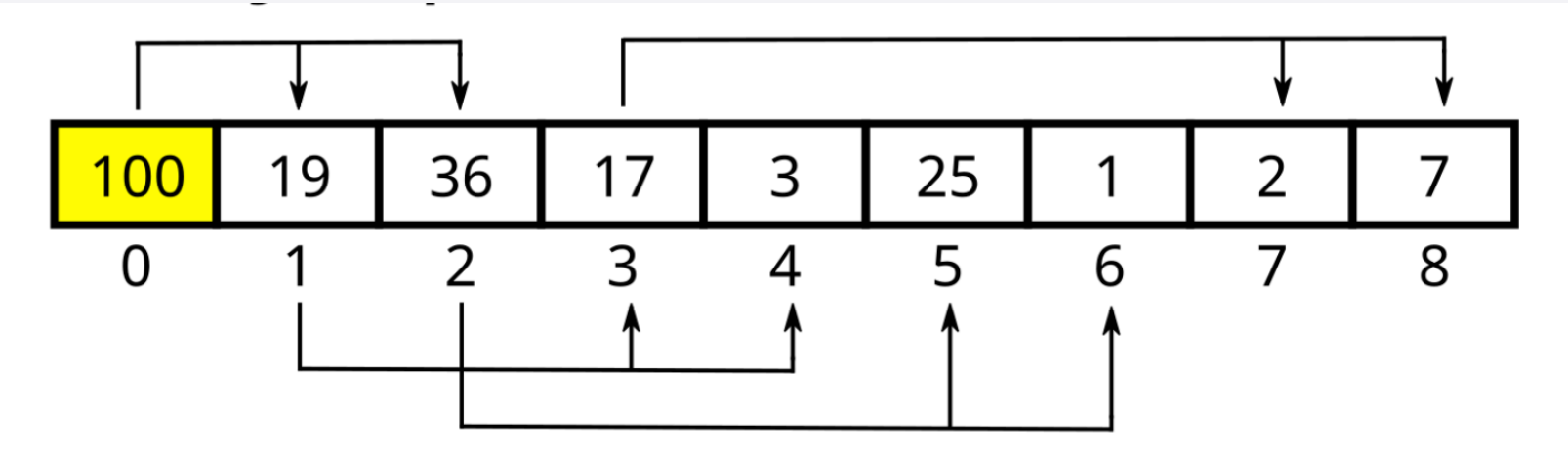
Jak przechowywać kopiec w tablicy?

Rozważmy kopiec *min-heap* – czyli drzewo o odpowiedniej strukturze.

Niech `A = []` będzie pustą tablicą. Umieszczamy wartości z wierzchołków w `A` w następujący sposób:

1. W `A[0]` umieszczamy korzeń.
2. Jeżeli wierzchołek jest umieszczony w tablicy pod indeksem `i`, to:

- jego lewe dziecko umieszczamy w `A` pod indeksem `j = 2 * i + 1`,
- jego prawe dziecko umieszczamy w `A` pod indeksem `j = 2 * i + 2`,
- jego rodzic jest umieszczony w `A` pod indeksem `j = (i - 1) // 2`.



Operacje na kopcu min-heap

push:

1. **Dodajemy element na koniec tablicy:**
   - Kopiec trzymamy w tablicy `A`.
   - Nowy element `x` dopisujemy na sam koniec: `A.append(x)`.
   - W drzewie odpowiada to wstawieniu `x` w **pierwsze wolne miejsce** na najniższym poziomie (od lewej).
2. **Sprawdzamy, czy własność kopca jest spełniona:**
   - W min-heap: `A[parent(i)] <= A[i]` dla każdego wierzchołka `i`.
   - Niech `i` będzie indeksem nowego elementu: `i := len(A) - 1`.
   - Niech `p := (i - 1) // 2` będzie indeksem jego rodzica.
3. **Podciągamy element w górę (sift-up):**
   - Jeśli `A[p] <= A[i]`, to kończymy – kopiec jest poprawny.
   - Jeśli `A[p] > A[i]`, to własność kopca jest złamana:
     - zamieniamy element z rodzicem: `A[p]` ↔ `A[i]`,
     - przechodzimy na pozycję rodzica: `i := p`,
     - powtarzamy od 2.
4. **Kończymy, gdy:**
   - dojdziemy do korzenia (`i == 0`) albo
   - rodzic nie jest większy od dziecka (`A[p] <= A[i]`).

pop:

1. **Zdejmujemy korzeń:**
   - W min-heapie najmniejsza wartość jest zawsze w korzeniu: `A[0]`.
   - Zapamiętujemy ją: `top := A[0]`.
2. **Wstawiamy ostatni element na górę:**
   - Zdejmujemy ostatni element tablicy: `last := A.pop()`.
   - Jeśli po tym tablica jest pusta, to zwracamy `top` i kończymy.
   - W przeciwnym razie ustawiamy: `A[0] := last`.
3. **Spychamy element w dół (sift-down):**
   - Niech `i := 0` będzie indeksem elementu, który właśnie trafił na górę.
   - Liczymy dzieci:
     - `l := 2 * i + 1` (lewe dziecko),
     - `r := 2 * i + 2` (prawe dziecko).
   - Wybieramy mniejsze z dzieci (bo min-heap):
     - jeśli `A[l] < A[r]`, to wybieramy `l`,
     - w przeciwnym razie wybieramy `r`.
4. **Naprawiamy własność kopca:**
   - Jeśli `A[i] <= A[child]`, to kończymy – kopiec jest poprawny.
   - Jeśli `A[i] > A[child]`, to własność kopca jest złamana:
     - zamieniamy `A[i]` ↔ `A[child]`,
     - schodzimy w dół: `i := child`,
     - Powtarzamy od 3.
5. **Zwracamy wynik:**
   - Po zakończeniu `sift-down` tablica `A` znów reprezentuje poprawny kopiec.
   - Zwracamy `top` – najmniejszy element, który był w korzeniu.

Kolejka priorytetowa

Kolejka priorytetowa to kolejka, gdzie każdy element ma przyporządkowaną liczbę oznaczającą **priorytet**.

Takiej kolejki możemy użyć na przykład do porządkowania zadań, które chcemy wykonać:

zadanie	priorytet
nauka do egzaminu z AiSD	1
odpowiedzieć na maila	3
sprzątanie mieszkania	5
zrobić zakupy	7

Taka kolejka to w zasadzie *interfejs dla kopca* – kopiec naturalnie porządkuje elementy wg wartości:

- `priority_queue.pop()` – zdejmujemy element o najwyższym priorytecie
- `priority_queue.push(`*sprzątanie mieszkania*, `3)` – kolejkujemy zadanie, a kopiec umieszcza je w odpowiednim miejscu.

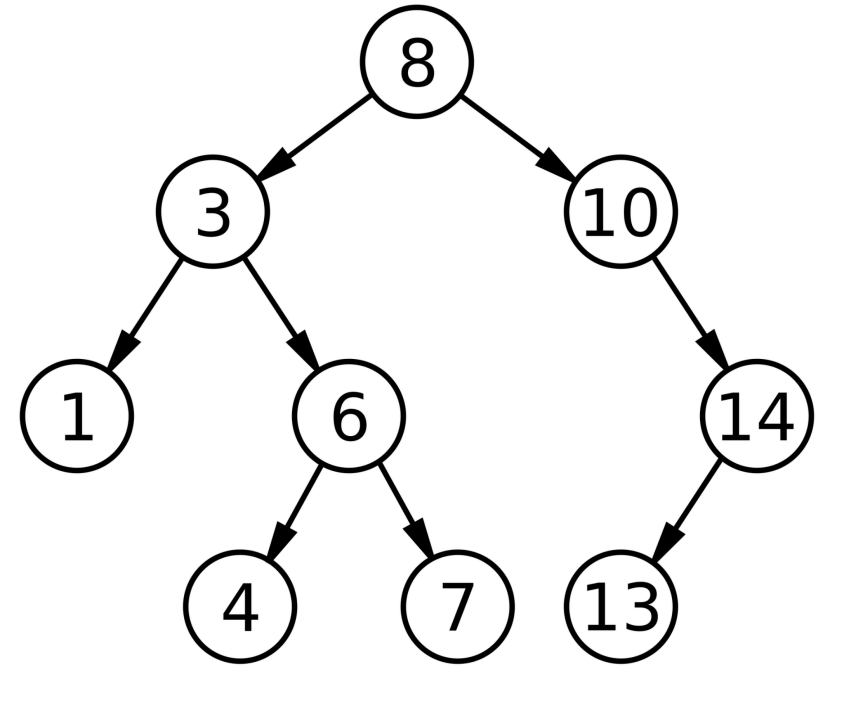
Sortowanie przez kopcowanie (*heapsort*) to jest dokładnie ten sam proces:

- Umieszczamy elementy do posortowania w kopcu (kolejka to tylko interfejs).
- Zdejmujemy elementy z kopca i układamy w liście.
- Wynik: posortowana lista elementów.

Binarne drzewa poszukiwań

Binarne drzewo poszukiwań (BST) to drzewo binarne, w którym:

- Każdy wierzchołek przechowuje wartość – klucz.
- Wszystkie klucze w lewym poddrzewie są mniejsze niż klucz rodzica.
- Wszystkie klucze w prawym poddrzewie są większe.



W konsekwencji możemy przeszukiwać je *tak jak słownik*.
Na przykład: Czy drzewo zawiera element o kluczu równym `x`?

1. Zaczynamy w korzeniu. Jeśli `key == x`, to kończymy.
2. Jeśli `key > x`, to powtarzamy procedurę dla lewego poddrzewa.
3. Jeśli `key < x`, to powtarzamy dla prawego poddrzewa.

Przechodzenie BST inorder

Inorder: przejście BST w kolejności **lewo** → **wierzchołek** → **prawo**.

1. Przechodzimy całe lewe poddrzewo.
2. Wypisujemy klucz dla korzenia.
3. Przechodzimy całe prawe poddrzewo.

W pseudokodzie moglibyśmy napisać tak:

```
INORDER(T: tree) {
    IF T IS NOT EMPTY:
        INORDER(T.LEFT)
        PRINT(T.KEY)
        INORDER(T.RIGHT)
}
```

Inorder wypisze wartości wszystkich kluczy w BST w kolejności rosnącej.

Temat

(12) Metoda z nawrotami i metoda zachłanna.
Problem n hetmanów, problem plecakowy, problem wyboru zajęć.

pozostawiam w 100% dla państwa.